

R 프로젝트의 이해

R 프로그래밍 with workflow design



R 프로젝트란?

- `.Rproj` 파일을 기반으로 RStudio에서 작업 디렉토리와 환경을 체계적으로 관리하는 기능
- 코드, 데이터, 결과물 등을 한 디렉토리 안에서 통합적으로 관리할 수 있음
- `.Rproj` 파일이 있는 디렉토리가 작업 디렉토리로 자동 설정되므로 `setwd()` 불필요
- 파일 경로를 **절대 경로** 대신 **상대 경로**로 지정 가능하므로 다른 컴퓨터나 환경에서도 동일하게 작동
- 대표적인 사용 예시
    - 과제 또는 연구 프로젝트 단위로 코드 관리
    - 보고서 자동화
    - 패키지 개발 등

R 프로젝트 생성 방법

- `RStudio > File > New Project`
- 선택 옵션
  - **New Directory**: 새 폴더 생성 후 프로젝트 초기화

    → `.Rproj` 파일이 자동 생성되며 디렉토리 구조를 설정할 수 있음
  - **Existing Directory**: 기존 폴더를 프로젝트로 전환

    → 기존 파일 구조 그대로 유지하며 `.Rproj` 추가
  - **Version Control**: Git 리포지토리 클론하여 시작

    → GitHub URL을 입력하여 원격 레포지토리를 클론(clone)하고 `.Rproj` 설정
- 옵션: `renv` 또는 Git 활성화 가능
  - `renv`를 선택하면 프로젝트마다 패키지 환경을 독립적으로 구성
  - Git을 선택하면 `.git` 폴더 생성 → Git 연동 및 버전 관리 가능

renv란?

- **renv (REproducible eNVironments)**: 프로젝트별 패키지 의존성과 버전을 분리하여 관리할 수 있게 해주는 R 패키지
- 주요 기능
  - 프로젝트마다 독립적인 패키지 라이브러리 구성
  - `renv.lock` 파일을 통해 패키지 목록과 버전 고정
  - 다른 컴퓨터나 사용자와 패키지 환경 공유 가능
- 주요 명령어
  - `renv::init()` – 새로운 renv 환경 초기화
  - `renv::snapshot()` – 현재 환경 상태 기록
  - `renv::restore()` – `renv.lock`에 기반해 환경 복원
- 활용 예시
  - 과거에 실행된 코드의 결과를 동일한 환경에서 재현하고 싶은 경우
  - 협업 시 동일한 분석 환경을 유지하고 싶은 경우

R 프로젝트의 장점

- **재현성 향상**: 상대경로 기반이므로 다른 컴퓨터나 사용자 환경에서도 코드 실행 가능
- **조직화된 작업 흐름**: 코드, 데이터, 결과물을 한 디렉토리 안에서 체계적으로 관리
  예시 디렉토리 구조
- **패키지 버전 관리**: `renv`를 활용하면 패키지 버전을 고정할 수 있어 의도한 대로 결과 재현 가능
- **협업 및 버전 관리**: Git과 함께 사용 시 팀원 간 충돌을 줄이고 변경 이력 추적 가능.
  특히 `.gitignore`로 불필요한 파일 제외 가능
- **작업 자동화에 유리**: Quarto / R Markdown 등과 결합하면 문서화와 보고서 자동화 가능

언제 R 프로젝트를 사용하는가?

- 다음과 같은 상황에서 R 프로젝트 사용 권장:
- 여러 개의 R 스크립트, 데이터셋, 결과물이 존재하는 경우
- 협업 중이거나 Git을 사용하는 경우
- Quarto, R Markdown, 패키지 개발 등 **중규모 이상의 작업**인 경우
- 간단한 실험용 코드에도 사용 가능하지만, **일정 규모 이상의 프로젝트에서는 필수에 가까움**
- 실제 사례로 본 책 집필 과정에서도 Quarto 기반 R 프로젝트 환경을 사용하여 챕터별 소스 파일, 이미지, 출력 결과를 일관되게 관리